

Torch-MLIR Model Zoo for SSLNPU Compiler Stack

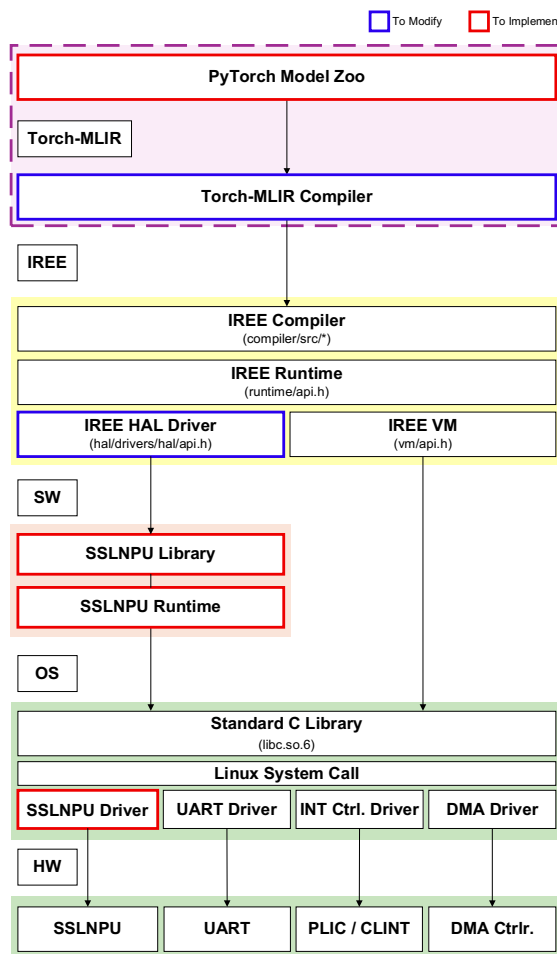
SSL 김병주

● 목적

- SSLNPU의 Compiler Stack 중 최상단인 Torch-MLIR의 원활한 구동을 위함
- Torch-MLIR의 Input (PyTorch) Model Zoo 구축

● 개요

- 해당 내용은 SSLNPU Compiler 구성에서 최상단(**보라색 점선**)에 위치해 있다.



- PyTorch Model Zoo를 구축함으로써 구현한 모델에 대한 MLIR 생성을 자동화할 수 있다.
- PyTorch Model Zoo를 확장해가며 여러 LLM 아키텍처에 대한 구현 및 수정을 용이하게 하도록 한다.

● 과정

1. Follow-ups: 각 Framework에 대한 전반적 이해

- A. Torch-MLIR (<https://github.com/llvm/torch-mlir>)
- B. IREE (<https://github.com/iree-org/iree>)
- C. AMD-SHARK-AI (<https://github.com/nod-ai/amd-shark-ai/tree/main>)
- D. Huggingface Transformers (<https://github.com/huggingface/transformers>)

2. 문제 상황 인식

- Torch-MLIR은 PyTorch 기반의 코드를 MLIR로 lowering을 진행.
- 범용 LLM Framework (Huggingface Transformers)은 PyTorch만을 사용해서 모델링하지 않음.
- LLM 모델링을 PyTorch 만을 사용한 소스코드로 작성해야 할 필요성

3. AMD-SHARK-AI 분석

- AMD-SHARK-AI에서 PyTorch 기반 LLM 모델링 코드를 Torch-MLIR을 사용해서 Lowering 하였고, 이를 오픈 소스로 공개
- 실제 LLM 모델링 코드 lowering 성공 사례로써 사용할 수 있음.
- AMD-SHARK-AI는 서버향 LLM 모델링, 이를 On-Device로 바꿔서 porting 하기 위해서 어떤 변화가 필요할지 분석. (e.g. PagedAttention 제거)

4. PyTorch 기반 LLM 부분 모델링

- Transformer의 기본적 연산들 (Attention, RMSNorm, MLP, Top-K 등)의 PyTorch 기반 구현 및 Torch-MLIR 적용.
- 해당 결과가 의도한 PyTorch 코드와 같은지 확인.
- AMD-SHARK-AI에서 미리 만든 것이 있다면 이를 사용해도 됨.

5. PyTorch 기반 LLM End-to-End 모델링 구현

- AMD-SHARK-AI를 참고하여 Torch-MLIR을 사용할 수 있는 LLM 모델링 코드 작성
- 모델의 MLIR 결과를 IR dumping을 통해 분석
- Target Model: Llama-3.2-1B(<https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct>)

6. Torch-MLIR Model Zoo 구축

- Target Model에 대한 확장 시도
- Llama-3.2-1B의 INT8 Quantized Model 사용 (<https://huggingface.co/unsloth/Llama-3.2-1B-Instruct-GGUF>, Q8_0)
- Whisper-tiny-INT8 모델링 (<https://huggingface.co/rhasspy/faster-whisper-tiny-int8>)